



12. Working with the Math and Random Classes

Exam Objectives

1. Use the Math class
2. Use the Random class

12.1 Using the Math class

The Java standard library contains `java.lang.Math` class, which contains methods for performing basic numeric operations such as the elementary exponential, logarithm, square root, and trigonometric functions. Observe that the `Math` class belongs to the `java.lang` package and since classes from this package are always accessible to all other class, there is no need to import this class in your code.

All the methods of this class are static. Thus, you don't need to instantiate an object of this class to use those method. In fact, it's constructor is private, so, you can't really create an instance using `new Math()`.

This class has a huge number of methods but that is only because it contains overloaded methods that are merely copies for various input types. For example, there are four overloaded `abs` methods. One each for `int`, `long`, `float`, and `double` parameter types - `int abs(int i)`, `long abs(long l)`, `float abs(float f)`, and `double abs(double d)`. Another characteristic that you may observe in the these four methods is that their return types match with their parameter types. All methods except the round methods follow this practice.

You need to be aware of the following methods of this class for the purpose of this exam:

1. **abs:** This method returns the positive value of the given argument. For example:

```
double d = Math.abs(-1.1); //assigns 1.1 to d
float f = Math.abs(-1.1); //will not compile because -1.1 is a double and so,
the return type will also be a double and a double cannot be assigned to a
float variable without a cast.
```
2. **ceil:** This method returns the smallest (closest to negative infinity) value that is greater than or equal to the argument and is equal to a mathematical integer. For example:

```
double d = Math.ceil(1.1); //assigns 2.0 to d
double d = Math.ceil(-1.1); //assigns -1.0 to d
```
3. **floor:** This method returns the largest (closest to positive infinity) value that is less than or equal to the argument and is equal to a mathematical integer. For example:

```
double d = Math.floor(1.1); //assigns 1.0 to d
double d = Math.floor(-1.1); //assigns -2.0 to d
```
4. **long round(double) and int round(float f):** Returns the closest long (or int, in the case of the float version) to the argument, with ties rounding to positive infinity. For example:

```
long d = Math.round(1.1); //assigns 1 to d
long d = Math.round(1.5); //assigns 2 to d
long d = Math.round(-1.1); //assigns -1 to d
long d = Math.round(-1.5); //assigns -1 to d
```

Observe the difference in the return values of `round(1.5)` and `round(-1.5)`.

The next method in the `Math` class that you need to know about is `double random()`.

The Math.random() method

This method returns a random double value greater than or equal to 0.0 and less than 1.0. For example:

```
double d = Math.random(); //assigns some value between 0.0 to less than 1.0 to d
```

Observe that it returns a double. So, if you want to use this method to get a random integer value between 0 to less than 10), you can do as follows:

```
int i = (int) (Math.random()*10); //assigns a random int value between 0 to 9 (both  
    inclusive) to i  
long l = Math.round(Math.random()*10); //Math.round(double) returns a long
```

Observe that multiplication by 10 gives you a double value between 0.0 to less than 10.0 and the cast simply truncates the decimal part. Also observe the parentheses around `Math.random()*10`. They are very important here. Recall from the Operators chapter that the cast operator has a higher precedence than the multiplication operator. Therefore, without the parentheses, return value of `Math.random()` will be cast to `int` first and that will always result in a zero because the value returned by the random method is always less than 1.0! Therefore, you need to first multiply by 10 to get a random double value between 0.0 to less than 10.0 and then truncate the decimal part by casting to get an `int` value between 0 to less than 10.

It is interesting that the exam asks you questions on the `Math.random()` method because Java has a dedicated feature rich class `java.util.Random` to generate random numbers, which is also on the exam.

12.2 Using the Random class

The `java.util.Random` class provides a more comprehensive, albeit somewhat complicated, solution to the problem of generating random numbers. Even the JavaDoc API description for this class accepts that many applications will find the `Math.random()` method simpler to use. However, for some reason, Oracle deems it important enough to test Java beginners on it.

The `Random` class lets you generate a stream of "pseudorandom" numbers, which means that the values generated by this class do not qualify as truly random but they are random enough for most practical purposes. The following example shows the basic usage of this class:

```
class TestClass{  
    public static void main(String[] args){  
        java.util.Random r = new java.util.Random(); //create an instance first  
        int i = r.nextInt(); //get a random int value  
        System.out.println(i);  
        double d = r.nextDouble(); //gets a random double value  
        System.out.println(d);  
    }  
}
```

Observe that, unlike with the `Math.random()` method, you need to first create an instance of the `Random` class and then call an appropriate `getXXX` method to get a random int, long, float, or double value.

Besides the no args `getInt` method, it also has `getInt(int bound)` method that takes a "bound". It returns a random integer between 0 to less than bound. For example, if you want to get a random integer between 0 to 9, you can do, `r.nextInt(10)`; Compare this with `Math.random()`, where you had to do `(int) (Math.random()*10)`; or `Math.round(Math.random()*10)`;

Using seed to create a Random instance

The `Random` class has two constructors - one that takes no argument, which you saw above and one that takes a long argument called "seed". Seed is a number that is required for the internal algorithm of the `Random` class to function. The random numbers generated by the algorithm depend on the seed. If two `Random` instances are created with the same seed value, then they will generate the same series of random numbers. For example, the following code prints the same two random numbers:

```
java.util.Random r1 = new java.util.Random(10);
java.util.Random r2 = new java.util.Random(10);
System.out.println(r1.nextInt()); //prints a random int
System.out.println(r2.nextInt()); //prints the same random int as above
```

The no-args constructor sets the seed of the random number generator to a value very likely to be distinct from any other invocation of this constructor. Therefore, unless you know what you are doing, it is best to use the no-args constructor instead of the one with the seed parameter.

The `Random` class also has a `setSeed(long)` method that resets the seed used to generate new random values. Although the exam doesn't test you on the details of the algorithm that generates random numbers, it may test you on the usage of this method. So, here is how it can be used:

```
class TestClass{
    public static void main(String[] args){
        java.util.Random r = new java.util.Random();
        int i = r.nextInt();
        r.setSeed(10000); //set the new seed
        i = r.nextInt(); //gets another
        System.out.println(d);
    }
}
```

The `Random` class shines when you need not just one or two random numbers but a series of random numbers. It has several methods that return a `Stream` of random numbers. The concept of streams is too advanced for this exam and so, I will not talk about in this book.

12.3 Exercise

1. Define a few double and int variables. Initialize them with positive as well as negative values. Assign the values of the double variables to the int variables after rounding them with the round, ceil, and floor methods. Observe which assignments require a cast and which do not.
2. Generate 10 random double values between 5 and 15(excluding) using `Math.random()`.
3. Generate 10 random double values between 1 and 10 (both including) using `Math.random()` and the `Random` class.
4. Create a array with a few elements. Shuffle the elements of the array in a random fashion.



13. Working with the String Class

Exam Objectives

1. Develop code that uses methods from the String class
2. Format Strings using escape sequences including %d, %n, and %s

13.1 Creating and manipulating Strings

13.1.1 What is a "string"?

In Java, a "string" is an object of class `java.lang.String`. It represents a series of characters. Strings such as "1234" or "hello" are really just objects of this class.

Not important for the JFCJA exam, but it is good to know that `String` is a **final** class. It extends `Object` and implements `java.lang.CharSequence` interface.

In the Java world, `String` objects are usually just called "**strings**". Although a string is just like any other regular object, it is such a fundamental object that Java provides special treatment to strings in terms of how they are created, how they are managed, and how they are used. Let's go over these aspects now.

13.1.2 Creating Strings

Creating strings through constructors

The `String` class has several constructors but for the purpose of the exam, you only need to be aware of the following:

1. **`String()`** - The no-args constructor creates an empty `String`.
2. **`String(String str)`** - Create a new `String` by copying the sequence of characters currently contained in the passed `String` or `StringBuilder` objects.
3. **`String(byte[] bytes)`** - Creates a new `String` by decoding the specified array of bytes using the platform's default charset.
4. **`String(char[] value)`** - Creates a new `String` so that it represents the sequence of characters currently contained in the character array argument.

Note that a string is composed of an array of chars. But that does not mean a string is the same as a char array. Therefore, you cannot apply the array indexing operator on a string. Thus, something like `char c = str[0];`, where `str` is a `String`, will not compile.

Creating strings through concatenation

The second common way of creating strings is by using the concatenation (, i.e., `+`) operator:

```
String s1 = "hello ";  
String s12 = s1 + " world"; //produces "hello world"
```

The `+` operator is overloaded in such a way that if either one of its two operands is a string, it converts the other operand to a string and produces a new string by joining the two. There is no restriction on the type of operands as long as one of them is a string.

The way `+` operator converts the non-string operand to a string is interesting:

1. If the non-string operand is a reference variable, the `toString()` method is invoked on that reference to get a string representation of that object.
2. If the non-string operand is a primitive variable or a primitive literal value, a wrapper object of the same type is created using the primitive value and then a string representation is obtained by invoking `toString()` on the wrapper object.
3. If the one of the operands is a null literal or a null reference variable, the string "null" is used instead of invoking any method on it.

The following examples should make this clear:

```
String s1 = "hello ";
String s11 = s1 + 1; //produces "hello 1"

String s12 = 1 + " hello"; //produces "1 hello"

String s2 = "" + true; //produces "true";

double d = 0.0;
String s3 = "" + d + "-"; //produces "-0.0-"

Object o = null;
String s4 = "hello " + o; //produces "hello null". No NullPointerException here.
```

Just like a mathematical expression involving the `+` operator, string concatenation is also evaluated from left to right. Therefore, while evaluating the expression `"1"+2+3`, `"1"+2` is evaluated first to produce `"12"` and then `"12"+3` is evaluated to produce `"123"`. On the other hand, the expression `1 + 2 + "3"` produces `"33"`. Since neither of the operands to `+` in the expression `1 + 2` is a `String`, it will be evaluated as a mathematical expression and will therefore, produce integer 3. `3 + "3"` will then be evaluated as `"33"`.

Remember that to elicit the overloaded behavior of the `+` operator, at least one of its operands must be a `String`. That is why, the following statements will not compile:

```
String x = true + 1; //Will not compile. First operand is boolean and second is int

Object obj = "string"; //OK
String y = obj + obj; //Will not compile. Even though obj points to a String at
    runtime, as far as the compiler is concerned, obj is an Object and not a String
```

Since the `toString` method is defined in the `Object` class, every class in Java inherits it. Ideally, you should override this method in your class but if you do not, the implementation provided by the `Object` class is used. Here is an example that shows the benefit of overriding `toString` in a class:

```
class Account{
    String acctNo;
    Account(String acctNo){
        this.acctNo = acctNo;
    }
}
```

```
//overriding toString
public String toString(){
    return "Account["+acctNo+"]";
}

public class TestClass{
    public static void main(String[] args){
        Account a = new Account("A1234");
        String s = "Printing account - "+a;
        System.out.println(s);
    }
}
```

The above code produces the following output with and without overriding toString:

```
Printing account - Account[A1234]
```

and

```
Printing account - Account@72bfaced
```

Observe that when compared to Object's toString, Account's toString generates a meaningful string. Since the Object class has no idea about what a class represents, it just returns a generic string consisting of the name of the class of the object, the at-sign character '@', and the unsigned hexadecimal representation of the hash code of the object. Don't worry, you will not be asked to predict this value in the exam. Just don't get scared if you see such a value in the exam.

On a side note, the print/println methods that we have often used also behave just like the + operator with respect to generating a string representation of the object that is passed to them. For example, when you call System.out.print(acct); where acct refers to an Account object, the print method invokes toString on that Account object and prints the returned string.

The += operator

In the chapter on operators, we saw that += is a compound operator. It applies the + operator on the two operands and then assigns the result back to the variable on the left side. As is the case with the + operator, the string concatenation behavior of += is triggered when the type of any one of its operands is String. Here is an example:

```
String s = "1";
s += 2; //expanded to s = s + 2;
System.out.println(s); //prints "12"
```

Furthermore, if the result of the += operator is a string, the type of the operand on the left must be something that can refer to a string, otherwise, the expression will not compile. There are only 4 such types other than String - the super classes of String, i.e., CharSequence and Object and, the interfaces that String implements, i.e., Serializable and Comparable. Here is an example:

```
int x = 1;  
x += "2"; //will not compile
```

Since the type of one of the operands in the above expression is `String`, the `String` concatenation behavior of `+=` will be triggered. However, the expression will not compile because you can't assign the resulting object of type `String` to a variable of type `int`.

Observe that if the type of the left operand is `String`, the type of the right operand can be anything because in that case, even if the type of the right operand is not `String`, it will be converted to a string as per the rules discussed above.

Now, can you tell what the following code will print?

```
Object m = 1;  
m += "2";  
System.out.println(m);
```

It will compile fine and print "12". First, 1 will be boxed into an `Integer` object, which will be assigned to `m`. This assignment is valid because an `Integer` "is-a" `Object`. Next, the expression `m += "2"` will be expanded to `m = m + "2"`. Since one of the operands of `+` in this expression is a string, a string concatenation will be performed, which will produce the string "12". This string will be assigned to `m`. The assignment is also valid because a `String` is an `Object`.

Okay, how about this?

```
Object m = "Hello ";  
m += 1;  
System.out.println(m);
```

It will fail to compile because as far as the compiler is concerned, type of `m` is `Object` and type of `1` is `int`. Therefore, when `m+=1` is expanded to `m = m + 1`, neither of the operands of `+` is a `String`!

Strings and Character Encodings

The following discussion is not required for the JFCJA exam but is important to know for any Java developer

You know that computers deal with just raw bytes containing zeros and ones. They have no idea about human readable characters. Thus, every human readable character needs to be converted into a series of zeros and ones, aka a binary representation, before it can be stored. Similarly, every stored character needs to be converted back from the series of zeros and ones to a human readable character. But what should a character, say 'A', look like in the storage? What series of zeros and ones should be used to represent 'A' or any other character? If you consider the number of languages that people speak, there are literally thousands of characters that need to have binary representations. Who decides the binary representation of these characters? It is not a trivial task. Because whatever representation you use to store your character data, must also be understood by other people who are trying to read the data that you have stored. The answer to these questions is "character encoding" or "charsets". Many computer scientists, groups, consortiums, and companies at various times have created "standards" that define how a particular character should be represented in the binary format. These standards are known as character encodings. One of the oldest and simplest standards is the ASCII standard which defines binary representation for all English alphabets and a few special characters using just 8 bits. So, for example, the character 'A' in ASCII can be stored as '1000001', which, in decimal is the number 65. So, if you store your text in ASCII format, you just need to tell the person reading that data that your data is in the ASCII format. They will then easily figure out that '1000001' means 'A'. However, the ASCII standard can store only 127 characters, which is enough to represent all English lower case and upper case characters but way too less than the total number of characters of all the languages that need to be represented!

Java uses a newer UTF-16 standard to store character data in Strings by default. UTF-16 uses up to 4 bytes to represent a character and is capable of representing a large number of characters. A good thing about this encoding is that its binary representations for all English letters matches with the ASCII representation, so, you will not notice a difference while dealing with English letters. You should be aware though that whenever you create a String, UTF-16 encoding is used to store the characters if you don't specify any encoding. Furthermore, if you want to read or write character data in any other charset, String class allows you to do that using the following two constructors - `String(byte[] bytes, String charsetName)` and `String(byte[] bytes, Charset charset)`. Similarly, you can use String's `getBytes(String charsetName)` and `getBytes(Charset charset)` methods to get a binary representation of text data in any other charset.

Strings containing special characters

You may sometimes need to store special characters such as new line, carriage return, and backslash characters in a String. Since these characters have a special meaning for a Java compiler, if you

try to put them in the String, they will cause compilation failure. For example, you can't write the following in your code:

```
String s = "first line  
second line"; //compilation failure
```

Java provides "escape sequences" to put such special characters in a String. An escape sequence starts with a backslash (\) followed by a letter. For example, if you want to write a new line character in a string, you should write \n, i.e., "first line\nsecond line". Other commonly used escape sequences are \r for carriage return character, \t for tab character, and \\ for the backslash character. Note that '\n' does not constitute two characters. It is just one character. So, "1\n2".length() will return 3 and not 4. You can use these escape sequence for specifying char values as well. For example, char tab = '\t';

13.1.3 String interning

Since strings are objects and since all objects in Java are always stored only in the **heap space**, (heap space is just a common space for storing all objects created by a program) all strings are stored in the heap space. However, Java keeps compile time constant strings (i.e., strings that the compiler knows to be constants) in a special area of the heap space called "**string pool**". Java keeps all other strings (including the ones created using the new keyword) in the regular heap space.

String interning

The purpose of the string pool is to maintain a set of unique strings. Any time you create a new string using a constant expression (i.e. using hardcoded String literals and String concatenation operations on constant String variables), Java checks whether the same string already exists in the string pool. If it exists, Java returns a reference to the same String object and if it does not, Java creates a new String object in the string pool and returns its reference. So, for example, if you use the string "hello" twice in your code as shown below, you will get a reference to the same string. We can actually test this theory out by comparing two different reference variables using the == operator as shown in the following code:

```
String str1 = "hello"; //1  
String str2 = "hello"; //2  
System.out.println(str1 == str2); //prints true //3  
String str3 = "hell"+"o";  
System.out.println(str1 == str3); //prints true //5  
String str4 = new String("hello");  
System.out.println(str2 == str4); //prints false //7  
String str5 = "world";  
String str6 = str2+str5; //creates "helloworld"  
String str7 = str2+str5; //creates "helloworld"  
System.out.println(str6 == str7); //prints false! //11
```

Recall from the Operators chapter that the == operator simply checks whether two references point to the same object or not and returns true if they do. In the above code, str2 gets the reference to the same String object which was created earlier at //1 because both str1 and str2

are assigned a compile time constant string "hello". Even `str3` gets the same reference because all the components of the expression `"hell"+"o"` (i.e., `"hell"` and `"o"`) are compile time constants themselves. That is why, `str1 == str2` and `str1 == str3` produce `true`. On the other hand, the string referred to by `str4` is a new string created at run time and so, `str4` gets a reference an entirely different String object. That is why, `str2 == str4` returns `false`.

In fact, when you do `new String("hello");` two String objects are created instead of just one if this is the first time the string "hello" is used in the anywhere in program - one in the string pool because of the use of a quoted string, and one in the regular heap space because of the use of `new` keyword.

Similarly, `str6` and `str7` get references to two different String objects because `str2` and `str5` are not constants. They are variables and so, the value of the expression `str5+str6` can only be computed at runtime. That is why `str6 == str7` returns `false`, even though both the variables point to strings containing the same data.

String pooling is Java's way of saving program memory by avoiding creation of multiple String objects containing the same value. It is possible to get a string from the string pool for a string created using the `new` keyword by using String's `intern` method. It is called **"interning"** of string objects. For example,

```
String str1 = "hello"; //str1 points to a String created in the string pool
String str2 = new String("hello"); //str2 points to a String created in the heap
String str3 = str2.intern(); //get an interned string referred to by str2
System.out.println(str1 == str3); //prints true
```

Generally, it is not possible to tell how the string returned by a method was created unless you go through the method code. The `intern` method is useful in such cases. If you want to work with an interned string only, you may get the interned version by invoking the `intern` method on the String reference returned by the method.

13.1.4 Quiz

Q. How many string objects are created in the following lines of code?

```
String a, b, c;
a = new String("Good Morning!");
b = a;
c = a + b;
```

Select 1 correct option.

- A. one
- B. two
- C. three
- D. four

Correct answer is C.

The statement `a = new String("Good Morning!");` creates two `String` objects - one containing "Good Morning!" in the string pool and one containing the same string data in the regular heap space. The statement `b = a;` does not create any new `String` object. The statement `c = a + b;` creates a new `String` object containing "Good Morning!Good Morning!". Thus, a total of three `String` objects are created.

13.1.5 String immutability

Strings are immutable. It is impossible to change the contents of a string once you have created it. Let me show you some code that looks like it is altering a string:

```
String s1 = "12";
s1 = s1+"34";
System.out.println(s1); //prints 1234
```

The output of the above code indicates that the string pointed to by `s1` has been changed from "12" to "1234". However, in reality, the original string that `s1` was pointing to remains as it is. A new string containing "1234" is created instead and its address is assigned to `s1`. So, after the last line of the above code, the JVM would have created three different strings - "12", "34", and "1234".

There are several methods in the `String` class that may make you believe that they change a string but just remember that a string cannot be mutated. Ever. Here are some examples:

```
String s1 = "ab";
s1.concat("cd");
System.out.println(s1); //prints "ab"
s1.toUpperCase();
System.out.println(s1); //prints "ab"
```

In the above code, `s1.concat("cd")` does create a new string containing "abcd" but this new string is not assigned to `s1`. Therefore, the first `println` statement prints "ab" instead of "abcd". The same thing happens with `toUpperCase()`. It does produce a new string containing "AB" but since this string is not assigned to `s1`, the second `println` statement prints "ab" instead of "AB".

13.1.6 Using the methods of the String class

The `String` class contains several methods that help you manipulate strings. To understand how these methods work, you may think of a string as an object containing an array of `chars` internally. These methods simply work upon that array. Since array indexing in Java starts with 0, any method that deals with the locations or positions of characters in a string, also uses the same indexing logic. Furthermore, any method that attempts to access an index that is beyond the range of this array throws `IndexOutOfBoundsException`.

Here are the methods that belong to this category with their brief `JavaDoc` descriptions:

1. `int length()` - Returns the length of this string.

For example, `System.out.println("0123".length());` prints 4. Observe that the index of the last character is always one less than the length.

2. `char charAt(int index)` - Returns the char value at the specified index. Throws `IndexOutOfBoundsException` if the index argument is negative or is not less than the length of this string.

For example, `System.out.println("0123".charAt(3));` prints 3.

3. `int indexOf(int ch)/indexOf(String s)/lastIndexOf(int ch)/lastIndexOf(String s)` - Returns the index within this string of the first (or last, in case of `lastIndexOf` methods) occurrence of the specified character (or String). Returns -1 if the character (or String) is not found.

Examples:

```
System.out.println("0123".indexOf('2')); //prints 2
System.out.println("Hi, How are you?".indexOf("are")); //prints 8
System.out.println("/user/ceo/MyClass.java".indexOf('/')); //prints 0
System.out.println("/user/ceo/MyClass.java".lastIndexOf("/")); //prints 9
System.out.println("/user/ceo/MyClass.java".lastIndexOf(".class")); //prints -1
```

The above code illustrates the use of the all four variations of the `indexOf` methods.

A design philosophy followed by the Java standard library regarding methods that deal with the starting index and the ending index is that the starting index is always inclusive while the ending index is always exclusive. String's substring methods works accordingly:

1. `String substring(int beginIndex, int endIndex)` - Returns a new string that is a substring of this string.

Examples:

```
System.out.println("123456".substring(2, 4)); //prints 34
```

Observe that the character at index 4 is not included in the resulting substring.

```
System.out.println("123456".substring(2, 6)); //prints 3456
```

```
System.out.println("123456".substring(2, 7)); //throws StringIndexOutOfBoundsException
```

2. `String substring(int beginIndex)` - This method works just like the other substring method except that it returns all the characters from `beginIndex` (i.e. including the character at the `beginIndex`) to the end of the string (including the last character).

Examples:

```
System.out.println("123456".substring(2)); //prints 3456
```

```
System.out.println("123456".substring(7)); //throws StringIndexOutOfBoundsException
```


The rule about not including the element at the ending index is followed not just by the methods of the String class but also by methods of other classes that have a concept of element positions such as `java.util.ArrayList`.

The following methods return a new String object with the required changes:

1. `String concat(String str)` - Concatenates the specified string to the end of this string.

Example - `System.out.println("1234".concat("abcd")); //prints 1 234abcd`

2. `String toLowerCase()/toUpperCase()` - Converts all of the characters in this String to lower/upper case. Example - `System.out.println("ab".toUpperCase()); //prints AB`

3. `String replace(char oldChar, char newChar)` - Returns a new string resulting from replacing all occurrences of `oldChar` in this string with `newChar`.

Example: `System.out.println("ababa".replace('a', 'c')); //prints cbcbc`

4. `String trim()` - Returns a copy of the string, with leading and/or trailing white space omitted.

Example:

```
System.out.println(" 123 ".trim()); //prints "123" (without the quotes)
```

One interesting thing about the String manipulation methods detailed above is that they return the same string if there is no change in the string as a result of the operation. Thus, all of the following print statements prints `true` because all of these operations return the same String object:

```
String s1 = "aaa"; //size of this string is 3
System.out.println(s1.substring(0,3) == s1); //prints true because the resulting
    substring is the same as the original string
System.out.println(s1.substring(0) == s1); //prints true because the resulting
    substring is the same as the original string
System.out.println(s1.replace('b', 'c') == s1); //nothing is replaced because there is
    no b in the string
System.out.println(s1.strip() == s1); //there is nothing to strip at the ends of the
    original string
```

It is very common to invoke these methods in the same line of code by chaining them together:

```
String str = "  hello ";
str = str.concat("world ").trim().concat("!").toUpperCase();
System.out.println(str);
```

The above code prints `HELLO WORLD!`. Note that such chaining is possible only because these methods return a String. You will see a similar chaining of methods in `StringBuilder/StringBuffer` classes as well.

Finally, here are a few methods that let you inspect the contents of a string:

1. `boolean startsWith(String prefix)`: Returns true if this string starts with the specified prefix.
2. `boolean endsWith(String suffix)`: Returns true if this string ends with the specified suffix.
3. `boolean contains(CharSequence s)`: Returns true if and only if this string contains the specified sequence of char values.
4. `boolean equals(Object anObject)`: Returns true if the contents of this string and the passed string are exactly same. Observe that the type of the parameter is `Object`. That's because this method is actually defined in the `Object` class and the `String` class overrides this method. So, you can pass any object to this method, but if that object is not a string, it will return false.
5. `boolean equalsIgnoreCase(String anotherString)`: Compares this `String` to another `String`, ignoring case considerations.

The above methods are fairly self-explanatory and work as one would expect after looking at their names, so, I am not going to talk about them in detail here but I suggest you take a look at their JavaDoc descriptions and write a few test programs to try them out. You will not get tricky questions on these methods in the exam.

The last method of the `String` class that you need to be aware of for the JFCJA exam is `String[] split(String regex)`. It splits this string into an array of strings around matches of the input. For example, `"/user/ceo/MyClass.java".split("/")` will return a `String` array containing three strings - "user", "ceo", and "MyClass.java". This method is very powerful because the argument is actually a "regular expression". Regular expressions is an advanced topic and is not on the JFCJA exam. The exam just expects you to know about the basic usage of this method.

13.1.7 Comparing strings

There are three ways you can compare strings - using the `==` operator and using the `equals` and the `compareTo` methods of the `String` class.

Comparison using the `==` operator

As I discussed earlier, the `==` operator, when applied to references, checks whether two references point to the same object or not. You can, therefore, use this operator on string references to check whether they point to the same `String` object or not. If the two references point to the same string, they are obviously "equal". The following code, for example, prints true for this reason:

```
String str = "hello";
System.out.println(str == "hello"); //prints true
```

Since creating strings in Java is quite easy, it is very tempting to use the `==` operator for testing their equality. However, it is very dangerous to do so as illustrated by the following code. All the `checkCode` method below wants to do is to check whether the string passed to it matches with the string referenced by the static variable named `code`.

```
public class TestClass{
    static String code = "1234";
    public static void checkCode(String str){
        System.out.println(code == str);
    }

    public static void main(String[] args){
        checkCode("1234");
        checkCode(new String("1234"));
    }
}
```

This code prints true for the first comparison and false for the second. Ideally, it should have printed true for both. The problem is that the `checkCode` method has no knowledge of how the string that was passed to it was created. As discussed earlier, when you create a string using the `new` keyword, an entirely new `String` object is created. That is why the second check returns false. It shows that `==` operator cannot guarantee you the right result if you want to compare the character data of two strings.

Comparison using the equals method

`String` class has an `equals` method that compares the actual character data of two strings to determine whether they are equal or not. This method returns true if the data matches and false otherwise. Since it compares the actual characters contained in two strings, it doesn't matter whether the two string references are references to the same `String` object or are references to two different `String` objects. Thus, `new String("1234").equals("1234")` will always return true even though both are two different `String` objects. The `equals` method, therefore, is a better way to compare two strings.

To fix the code shown above, just replace `System.out.println(code == str);` with `System.out.println(code.equals(str));` It is common to invoke `equals` on a `String` literal and pass a `String` variable instead of the other way round., i.e., `"1234".equals(str);` is preferred to `str.equals("1234")` because if `str` is null, the first style returns false but the second style throws a `NullPointerException`.

Comparison using the compareTo method

While `==` and `equals` tell you only about the equality of two `Strings`, the `compareTo` method tells you about their lexicographical order, also known as dictionary order, as well. The `compareTo` method has the following signature:

```
int compareTo(String anotherString)
```

The `int` value that it returns tells you whether `anotherString` is "lessor" or "greater", i.e., whether

anotherString should appear before or after this String in a dictionary. If the two Strings are equal, the return value is 0. For example:

```
String a = "alice";
String b = "bob";
String c = "charlie";
System.out.println(a.compareTo(b)); //prints -1
System.out.println(b.compareTo(a)); //prints 1
System.out.println(a.compareTo(c)); //prints -2
System.out.println(a.compareTo(new String("alice"))); //prints 0
```

A negative value indicates that this String is lesser than anotherString. Furthermore, the magnitude of the returned number tells you the difference of the two character values at the earliest position where the two Strings differ. For example, "alice" and "charlie" differ at the first position itself and the difference between the two characters at the first position, i.e., 'a' and 'c' is 2. Since 'a' comes before 'c', the return value of a.compareTo(c) is -2.

13.1.8 Quiz

Q. What will the following code print?

```
String a1 = "alice";
String a2 = new String(a1);
System.out.print( (a1==a2)+" "+a1.equals(a2)+" "+a1.compareTo(a2));
```

Select 1 correct option.

- A. false false false
- B. true false 1
- C. false true true
- D. false true 0

Correct answer is D.

new String(a1) creates a new String object. Although the new object will contain the same characters as the String pointed to by a1, it is, nevertheless, a different object. Therefore, a1==a2 will return false.

Since equals and compareTo compare the contents of the two String objects and since the contents of the Strings pointed to be a1 and a2 are the same, a1.equals(a2) will return true and a1.compareTo(a2) will return 0.

13.2 Formatting Strings

So far you have used System.out.print and System.out.println methods to print messages to the console. For example, if you want to print something like "Hello, Bob! How are you? To-

day is 1st Oct 2020.", you could do - `System.out.print("Hello, "+name+"! How are you? Today is "+dateString+".");` where the value for name would be taken from the name variable and the value for date would be taken from the dateString variable. The problem with this style is quite apparent. It is fine for printing short messages during development but it is too primitive when you need to print large preformatted messages with lots of parameterized data.

A better way to achieve the same is: `System.out.format("Hello, %s! How are you? Today is %s.", name, dateString);` Observe that the message body is now separated from the customized values appearing inside the message. The resulting string is produced by replacing the first `%s` with the value of name variable and the second `%s` with the value of the dateString variable.

Recall that `out` is a static variable in the `java.lang.System` class of type `java.io.PrintStream`. `PrintStream` has the following `format` method, which makes this possible:

`PrintStream format(String format, Object... args)` : It writes a formatted string to this output stream using the specified format string and arguments. The first parameter is the main body of the text that you want to print (also called "template") and the second is a varargs parameter for the values that you want to embed into the message. Each value is formatted and inserted according to the format specifications contained within the template to create the resulting String.

`PrintStream` also has a `printf` method, which works exactly the same as `format`. You may see the usage of both of them in the exam questions.

The format specification itself is quite powerful and has several bells and whistles for generating complicated messages. However, for the purpose of the exam, you need to know only the basic technique using a few format specifiers: `%s`, `%d`, and `%n`.

1. `%s` : Used to embed String values.
2. `%d` : Used to embed numeric values in the format.
3. `%n` : Used to embed the line character

Remember that it must be possible for the values to be converted into the given format. For example, if your format specifier is `%d`, then the corresponding value must be an integral value (byte, char, short, int, and long). If you try to pass any other type of value, such as a double or a String, a `java.util.IllegalFormatConversionException` will be thrown. However, any type of value can be converted into a String, therefore, you can use any type of value with `%s`. Furthermore, the number of format specifiers used in the format String must not be more than the number of arguments in the varargs array, otherwise a `java.util.MissingFormatArgumentException` will be thrown. Extra values are fine.

The following example and its output should make this clear:

```
String name = "bob";
int acctNo = 1234;
int points = 100;
System.out.format("Hello, %s!\n\nThe point balance in your account %s is %d.%n",
    name, acctNo, points); //4
```

```
System.out.format("Great Job!\n", name); //5
System.out.format("These points are worth %d USD.%nThank you!", points*10.0); //6
```

It produces the following output:

```
The point balance in your account 1234 is 100.
Great Job!
These points are worth Exception in thread "main"
    java.util.IllegalFormatConversionException: d != java.lang.Double
```

You should observe the following points about the above code:

1. The line at //4 shows that integral values can be embedded in the message with %d as well as %s.
2. The line at //5 shows that extra values is not a problem.
3. The line at //6 shows that a double value cannot be converted to an integer using %d.

The String.format method

Sometimes you may not necessarily want to print the formatted string but use it for a different purpose. For example, you may want to use them as the body of an email or you may just want to store them in the database. The `String` class has a static method named `format`, which generates formatted Strings using the same formatting principles explained above. But instead of printing the resulting string, it simply returns the generated string. For example: `String message = String.format("Hello, %s!", name);`

13.3 Exercise

1. Create a few strings that contain special characters such as new line and backslash. Print their lengths.
2. Use the `indexOf` method to print the positions at which the strings created in the previous task contain the special characters.
3. Write a method that accepts a `String` as input and count the number of spaces in the string from start to until it finds an 'x', or if there is no 'x' in the string, till end.
4. Write code to determine whether the `toString` and `substring` methods of `String` class return an interned string or not. Confirm your results by checking the JavaDoc API descriptions of these methods.
5. Write a method that takes a `String` and returns a `String` of the same length containing the 'X' character in all positions except the last 4 positions. The characters in the last 4 positions must be the same as in the original string. For example, if the argument is "12345678", the return value should be "XXXX5678".

6. Write a method that takes a `String[]` as an argument and returns a `String` containing the concatenation of all the strings in the input array. Invoke your method with different arguments. Make sure that your code handles the cases where the argument is null, contains a few nulls, or contains only nulls.
7. What will the following code print and why?

```
String s = "a";
String[] sa = { "a", s, s.substring(0, 1), new String("a"), "'+'a' " };
for(int i=0; i<sa.length; i++){
    System.out.println(i);
    System.out.println(s == sa[i]);
    System.out.println(s.equals(sa[i]));
}
```

8. Create two arrays. One containing `String` values: Amy, Boyd, and Cathy, and another one containing `int` values: 10, 11, and 12. Use `printf` and `format` methods to create an HTML table showing these values as name value pairs.
